LLM-Powered Smart Contract Auditor
Team: Adil, Ulan Mustafin

1. Research Gap

Currently LLM technology is wide-spread and is a generally big trend in computer science. The implications of the use of Large Language Models for Smart Contract vulnerability detection is an interest of our project. We found the research studies that investigate the application of LLM for the security purposes: direct prompting, integration and program analysis with auditing models [1-5]. Existing tools and frameworks for analysis are providing the approaches and benchmarks to assess vulnerabilities [6-8, 11]. However, the good benchmark performance does not guarantee reliability of the smart contract. In our project we are focusing on four evaluation concerns.

1.1 Training data contamination
Public datasets support replicable evaluation, but the fact that their contract appeared in LLM's training data may affect the results of its responses. Classification could be "poisoned" by the LLM's initial training data and do not show its understanding of the vulnerability, but rather just familiarity with it [22]. Thus we will compare the LLM public examples with the locally created ones. It will reveal sensitivity to changes and understanding of the underlying principle of the vulnerability.

1.2 False alarms on closely matched safe contracts
Our auditor should be able to recognize the fact and the absence of the vulnerability [1, 2]. So that we can be sure that we do not get False Positive or False Negative results. Thus we will test the fixed from the vulnerabilities contract and look whether the LLM keeps flagging it as vulnerable.

1.3 Compiler version-aware arithmetic detection
Solidity before v.0.8. arithmetics worked a little different and allowed integer wrapping. Later versions introduced overflow and underflow checks. We will test the model - whether or not it will recognize this fact, since it is an established security concern [13, 18].

1.4 Sensitivity to prompt design
There are two main ways to prompt - few-show and step-by-step [20, 21]. We will test these two approaches to measure the effect of choosing either of the designs. We will compare definitions, instructions and differences between the results.

2. Research Questions

- Research Question 1: How well does LLM detect and recognize the reetrancy and integer overflow or underflow, measuring the precision and recall?
- Research Question 2: How does changing the prompt design affect detection performance?
- Research Question 3: Can LLM distinguish between vulnerable and safe contract?
- Research Question 4: Does model adjust to the version of the Solidity compiler?

3. Planned Evaluation and Contribution

We will evaluate 50 labeled contracts with at least 15 of them being safe contracts. Then we will use public examples from SmartBugs with repaired ones [8]. We will test the same contracts using different prompt design to independently measure the difference between them. Also, the deliverables will include a pipeline, labeled dataset, table of comparison and error analysis. We won't train a new model, but evaluate existing ones.